

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №19

Тема: Створення алгоритмів на обчислення

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Навчитися розробляти алгоритми обчислення за формулами: застосовувати модуль `math`, враховувати порядок виконання операцій, правила округлення та обмеженість точності дійсних чисел; дослідити похибки обчислень з плаваючою комою та способи їх контролю.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Обчислювальний алгоритм — алгоритм, основу якого становить виконання арифметичних операцій за формулами. Перед програмуванням складну формулу розбивають на **прості кроки** з проміжними змінними: це зменшує кількість помилок, спрощує трасування та дозволяє повторно використовувати проміжні результати.

2. Пріоритет операцій у Python (від найвищого до найнижчого): `**` → унарні `+` `-` → `*` `/` `//` `%` → `+` `-`. Операції однакового пріоритету виконуються зліва направо (окрім `**`, який виконується справа наліво: `2 ** 3 ** 2 = 2 ** 9 = 512`). Дужки змінюють порядок виконання. Типова помилка: формулу `a / b * c` записують як `a / b * c`, хоча потрібно `a / (b * c)`.

3. Модуль `math` містить математичні функції:

Функція	Призначення	Функція	Призначення
<code>sqrt(x)</code>	квадратний корінь	<code>sin(x), cos(x)</code>	тригонометричні (x у радіанах)
<code>pow(x, y)</code>	ступінь x^y	<code>radians(g)</code>	градуси → радіани
<code>exp(x)</code>	e^x	<code>log(x), log10(x)</code>	натуральний і десятковий логарифми
<code>floor(x), ceil(x)</code>	округлення вниз / вгору	<code>hypot(a, b)</code>	$\sqrt{a^2 + b^2}$
<code>pi, e</code>	константи π і e	<code>isclose(a, b)</code>	порівняння з допуском

4. Округлення. Функція `round(x, k)` округлює до `k` знаків після коми (у Python 3 — за правилом «округлення до парного»: `round(0.5) = 0`, `round(1.5) = 2`, `round(2.5) = 2`). `math.floor` округлює завжди вниз, `math.ceil` — вгору. Для виведення з фіксованою кількістю знаків використовують форматування `f"{x:.3f}"`.

5. Похибки дійсних чисел. Дійсні числа зберігаються у двійковому вигляді (стандарт IEEE 754, тип `float` — 53 значущі двійкові розряди, ≈ 15 – 16 десяткових цифр). Більшість десяткових дробів (наприклад, `0,1`) не має точного двійкового представлення,

тому $0.1 + 0.2 \neq 0.3$. Наслідки: (а) дійсні числа **не можна порівнювати на рівність** оператором `==` — використовують `math.isclose(a, b)` або `abs(a - b) < eps`; (б) при відніманні близьких чисел відбувається **втрата значущих цифр**; (в) для точних десяткових обчислень (гроші) застосовують модуль `decimal`.

6. Абсолютна й відносна похибка. Якщо x – точне значення, а x^* – наближене, то абсолютна похибка $\Delta = |x - x^*|$, відносна похибка $\delta = \Delta / |x|$. Саме відносна похибка показує якість наближення незалежно від масштабу величини.

Хід роботи

Завдання 1. Ручне обчислення виразів із урахуванням порядку операцій

Обчислити вручну значення виразів Python, явно вказавши порядок виконання операцій, а потім перевірити відповіді за допомогою інтерпретатора.

```
1) 2 + 3 * 4 ** 2 / 8 - 1
2) (2 + 3) * 4 ** 2 // 8 % 5
3) -3 ** 2 + 2 ** 3 ** 2 // 100
4) 17 // 5 * 5 + 17 % 5
5) 10 / 4 * 2 - 10 // 4 * 2
```

Порядок виконання:

1. Для кожного виразу розставте номери операцій у порядку їхнього виконання (з урахуванням пріоритетів і правила «зліва направо»).
2. Виконайте обчислення по кроках, записуючи проміжні результати (наприклад, $4 ** 2 = 16$; $3 * 16 = 48$; ...).
3. Зверніть увагу на тип результату: де буде `int`, а де `float` (операція `/` завжди дає `float`).
4. Запишіть остаточні значення в таблицю «вираз — проміжні кроки — результат — тип».
5. Перевірте відповіді в інтерпретаторі Python; якщо результат не збігся — знайдіть, де порушено порядок операцій.
6. Запишіть вираз 4) у загальному вигляді для довільних a , b і поясніть, яку тотожність він ілюструє.

Завдання 2. Трасування алгоритму обчислення площі трикутника за формулою Герона

За трьома сторонами a , b , c трикутника знайти площу S , периметр P та радіус описаного кола R за формулами: $p = (a + b + c) / 2$; $S = \sqrt{p(p-a)(p-b)(p-c)}$; $R = abc / (4S)$.

Порядок виконання:

1. Розбийте обчислення на прості кроки: напівпериметр $p \rightarrow$ добуток під коренем $q \rightarrow$ площа $S \rightarrow$ радіус R . Запишіть псевдокод цих кроків.
2. Побудуйте таблицю трасування зі стовпцями «крок», «змінна», «формула», «значення» для сторін $a = 13$, $b = 14$, $c = 15$.
3. Обчисліть вручну: p , $p - a$, $p - b$, $p - c$, q , S (результат має бути цілим числом – перевірте добуток $21 \cdot 8 \cdot 7 \cdot 6$), потім R .
4. Повторіть трасування для $a = 3$, $b = 4$, $c = 5$ (прямокутний трикутник); перевірте S за формулою $S = ab/2$ та переконайтеся, що $R = c/2$.

5. Виконайте трасування для $a = 1$, $b = 2$, $c = 5$ і з'ясуйте, що станеться з виразом під коренем; зробіть висновок про необхідність перевірки вхідних даних (нерівність трикутника).

Завдання 3. Програмна реалізація обчислювальних алгоритмів

Реалізувати мовою Python обчислення за формулами з використанням модуля `math` та перевірити результати ручними розрахунками.

Порядок виконання:

1. Напишіть функцію `triangle(a, b, c)`, що повертає кортеж (P, S, R) за формулою Герона. Якщо сторони не утворюють трикутник (порушена нерівність трикутника), функція повертає `None`.
2. Напишіть функцію `compound(p0, rate, years, n)`, що обчислює суму вкладу за формулою складних відсотків $S = P_0 \cdot (1 + r/n)^{(n \cdot t)}$, де r — річна ставка (частка), n — кількість нарахувань на рік.
3. Напишіть функцію `hypot_angle(a, b)`, що за катетами повертає гіпотенузу та гострий кут $\alpha = \arctan(b/a)$ у градусах (використайте `math.hypot`, `math.atan`, `math.degrees`).
4. Напишіть функцію `cylinder(r, h)`, що повертає об'єм $V = \pi r^2 h$ та площу повної поверхні $S = 2\pi r (r + h)$.
5. Перевірте функції на даних із ручних розрахунків і виведіть результати з потрібною кількістю знаків після коми.

Приклад реалізації мовою Python:

python

```
import math

def triangle(a, b, c):
    if a + b <= c or a + c <= b or b + c <= a:
        return None
    p = (a + b + c) / 2
    s = math.sqrt(p * (p - a) * (p - b) * (p - c))
    r = a * b * c / (4 * s)
    return a + b + c, s, r

def compound(p0, rate, years, n):
    return p0 * (1 + rate / n) ** (n * years)

def hypot_angle(a, b):
    return math.hypot(a, b), math.degrees(math.atan(b / a))

def cylinder(r, h):
    return math.pi * r ** 2 * h, 2 * math.pi * r * (r + h)

for sides in ((13, 14, 15), (3, 4, 5), (1, 2, 5)):
    res = triangle(*sides)
    if res is None:
        print(f"Сторони {sides}: трикутник не існує")
    else:
        print(f"Сторони {sides}: P = {res[0]}, S = {res[1]:.2f}, R = {res[2]:.3f}")

for n in (1, 12, 365):
    print(f"Вклад 10000 грн, 12% річних, 5 років, n = {n:>3}: {compound(10000, 0.12, 5, n):.2f} грн")

c, angle = hypot_angle(3, 4)
```

```
print(f"Катети 3 і 4: гіпотенуза = {c:.1f}, кут = {angle:.2f}°")
v, s = cylinder(2, 5)
print(f"Циліндр r = 2, h = 5: V = {v:.3f}, S = {s:.3f}")
```

Результат виконання програми:

```
Сторони (13, 14, 15): P = 42, S = 84.00, R = 8.125
Сторони (3, 4, 5): P = 12, S = 6.00, R = 2.500
Сторони (1, 2, 5): трикутник не існує
Вклад 10000 грн, 12% річних, 5 років, n = 1: 17623.42 грн
Вклад 10000 грн, 12% річних, 5 років, n = 12: 18166.97 грн
Вклад 10000 грн, 12% річних, 5 років, n = 365: 18219.39 грн
Катети 3 і 4: гіпотенуза = 5.0, кут = 53.13°
Циліндр r = 2, h = 5: V = 62.832, S = 87.965
```

*Висновок за результатами: для трикутника (13, 14, 15) площа дорівнює **84.00**, радіус описаного кола 8.125 — це збігається з ручним трасуванням; для прямокутного трикутника (3, 4, 5) $R = 2.5 = c/2$. Для сторін (1, 2, 5) нерівність трикутника порушена, тому функція повертає None замість того, щоб викликати sqrt від від'ємного числа. Збільшення частоти нарахування відсотків n підвищує підсумкову суму вкладу, але зростання швидко сповільнюється й наближається до межі $P_0 \cdot e^{(r \cdot t)}$ — різниця між щомісячним і щоденним нарахуванням вже невелика.*

Завдання 4. Дослідження похибок обчислень із плаваючою комою

Експериментально дослідити обмеженість точності типу float та способи коректного порівняння й обчислення.

Порядок виконання:

1. Виведіть значення виразів $0.1 + 0.2$, $0.1 + 0.2 == 0.3$, $0.1 * 3$ та $\text{round}(0.1 + 0.2, 10) == 0.3$; поясніть результати.
2. Перевірте порівняння з допуском: $\text{math.isclose}(0.1 + 0.2, 0.3)$ та $\text{abs}(a - b) < 1e-9$.
3. Складіть у циклі 10 разів число 0.1 і порівняйте суму з 1.0; обчисліть абсолютну похибку результату.
4. Дослідіть втрату значущих цифр: обчисліть вираз $(1 - \cos x) / x^2$ для $x = 1e-4, 1e-6, 1e-8$ прямо та за еквівалентною формулою $2 \cdot \sin^2(x/2) / x^2$; точне значення границі при $x \rightarrow 0$ дорівнює 0,5.
5. Виконайте ті самі обчислення $0.1 + 0.2$ за допомогою decimal.Decimal і переконайтеся, що вони точні.
6. Обчисліть для $x = 0.1 + 0.2$ абсолютну та відносну похибку відносно 0.3 і запишіть їх у науковому форматі.

Приклад реалізації мовою Python:

python

```
import math
from decimal import Decimal

print("0.1 + 0.2 =", 0.1 + 0.2)
print("0.1 + 0.2 == 0.3 ?", 0.1 + 0.2 == 0.3)
print("isclose:", math.isclose(0.1 + 0.2, 0.3), "; |a-b| < 1e-9:", abs(0.1 + 0.2 - 0.3) < 1e-9)

s = 0.0
for _ in range(10):
    s += 0.1
print("10 разів по 0.1 =", s, "; s == 1.0 ?", s == 1.0, "; похибка =", abs(1.0 - s))
```

```

print("\n      x          пряма формула          стійка формула")
for x in (1e-2, 1e-4, 1e-6, 1e-8):
    direct = (1 - math.cos(x)) / x ** 2
    stable = 2 * math.sin(x / 2) ** 2 / x ** 2
    print(f"{x:8.0e}      {direct:16.12f}      {stable:16.12f}")

print("\nDecimal:", Decimal("0.1") + Decimal("0.2"), "; == 0.3 ?", Decimal("0.1") +
Decimal("0.2") == Decimal("0.3"))
a, exact = 0.1 + 0.2, 0.3
print(f"Абсолютна похибка = {abs(a - exact):.2e}; відносна = {abs(a - exact) /
exact:.2e}")

```

Результат виконання програми:

```

0.1 + 0.2 = 0.300000000000000004
0.1 + 0.2 == 0.3 ? False
isclose: True ; |a-b| < 1e-9: True
10 разів по 0.1 = 0.9999999999999999 ; s == 1.0 ? False ; похибка =
1.1102230246251565e-16

      x          пряма формула          стійка формула
1e-02      0.499995833347      0.499995833347
1e-04      0.499999996961      0.499999999583
1e-06      0.500044450291      0.500000000000
1e-08      0.000000000000      0.500000000000

Decimal: 0.3 ; == 0.3 ? True
Абсолютна похибка = 5.55e-17; відносна = 1.85e-16

```

Висновок за результатами: вираз $0.1 + 0.2 == 0.3$ хибний, оскільки сума дорівнює 0.300000000000000004: числа 0,1 і 0,2 не мають точного двійкового представлення. Порівняння за допомогою `math.isclose` або допуску дає коректну відповідь, а обчислення в `Decimal` – точні. Пряма формула $(1 - \cos x)/x^2$ при зменшенні x катастрофічно втрачає точність через віднімання близьких чисел (при $x = 1e-8$ вона дає 0.0 замість 0,5), тоді як алгебраїчно еквівалентна стійка формула зберігає всі значущі цифри. Звідси практичне правило: алгоритм обчислення слід обирати з урахуванням чисельної стійкості, а не лише математичної рівності формул.

Контрольні питання

1. У чому полягає суть обчислювального алгоритму? Чому складні формули розбивають на прості кроки?
2. Який пріоритет мають операції в Python? Чому $-3 ** 2$ дорівнює -9 , а не 9 ?
3. Які функції модуля `math` ви використовували? Як перевести градуси в радіани?
4. Як працює функція `round()` у Python 3? Чим `floor` відрізняється від `ceil`?
5. Чому $0.1 + 0.2 != 0.3$? Як коректно порівнювати дійсні числа?
6. Що таке абсолютна й відносна похибка? Яка з них краще характеризує точність і чому?
7. Що таке втрата значущих цифр і за яких умов вона виникає? Наведіть приклад.
8. У яких випадках для обчислень доцільніше застосовувати модуль `decimal`, ніж тип `float`?

Зміст звіту

1. Тема, мета роботи.
2. Ручні обчислення виразів та таблиця трасування для формули Герона (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Пояснення результатів дослідження похибок.
5. Висновки.